

Project Work Data Mining

Marika Rucco

Settembre 2026

Contents

1	Introduzione e Motivazione	3
2	Dataset	3
2.1	Caratteristiche generali dei dataset	4
2.2	Mushroom	4
2.3	Retail	5
2.4	PUMSB	6
2.5	Kosarak	6
2.6	Confronto tra i dataset	7
3	Algoritmi	7
3.1	Cellular Learning Automata	7
3.1.1	Classi	8
3.1.2	Fase 1: individuazione degli itemset frequenti di dimensione 1	8
3.1.3	Fase 2: pruning e compressione del dataset	8
3.1.4	Fase 3: costruzione delle celle e delle liste di prossimità	9
3.1.5	Fase 4: pruning delle liste di prossimità e estrazione degli itemset frequenti	9
3.2	Apriori	10
3.2.1	Fase 1: individuazione degli itemset frequenti di dimensione 1	10
3.2.2	Fase 2: generazione dei candidati	10
3.2.3	Fase 3: verifica della proprietà di antimonotonicità	10
3.2.4	Fase 4: calcolo del supporto dei candidati ed estrazione degli itemset frequenti	11
3.3	FP-Growth	11
3.3.1	Fase 1: costruzione dell'FP-Tree	11
3.3.2	Fase 2: estrazione degli itemset frequenti	12

4	Analisi dei risultati	13
4.1	Dataset Mushroom	14
4.2	Dataset Retail	15
4.3	Dataset Pumsb	16
4.4	Dataset Kosarak	17
4.5	Confronto complessivo	19

1 Introduzione e Motivazione

L'obiettivo principale di questo progetto è l'implementazione e la valutazione sperimentale del metodo proposto da Sohrabi e Roshani nell'articolo Frequent Itemset Mining Using Cellular Learning Automata [1]. Il problema del frequent itemset mining consiste nell'individuare, all'interno di un insieme di transazioni, gli insiemi di item che ricorrono con una frequenza superiore a una determinata soglia, detta minimum support (minsup). Tale problema riveste particolare interesse nell'ambito dell'analisi dei dati e dell'estrazione di informazioni, soprattutto nel caso di dataset di grandi dimensioni. Gli autori dell'articolo propongono un approccio basato sui Cellular Learning Automata (CLA), con l'obiettivo di affrontare il problema del frequent itemset mining attraverso un paradigma differente rispetto agli algoritmi tradizionali. In particolare, il metodo viene confrontato con algoritmi di riferimento, mostrando sperimentalmente risultati competitivi in termini di tempo di esecuzione. In questo progetto viene implementato il metodo basato sui Cellular Learning Automata e ne vengono analizzate le prestazioni attraverso una serie di esperimenti. Per valutare il comportamento dell'approccio proposto, i risultati vengono confrontati con quelli ottenuti mediante due algoritmi alternativi per il frequent itemset mining. Gli esperimenti vengono inoltre eseguiti su dataset differenti e considerando diverse soglie di minsup, in modo da analizzare l'influenza di tali variabili sulle prestazioni degli algoritmi. Nelle sezioni successive verranno descritti i dataset utilizzati e le relative caratteristiche, verranno presentati i dettagli implementativi degli algoritmi considerati e verrà illustrato il protocollo sperimentale adottato. Infine, saranno discussi e confrontati i risultati ottenuti, con particolare attenzione ai tempi di esecuzione e al numero di frequent itemset individuati, per concludere con una valutazione complessiva degli approcci analizzati.

2 Dataset

Per la valutazione sperimentale degli algoritmi sono stati utilizzati quattro dataset: Mushroom, Retail, PUMSB e Kosarak. Tutti i dataset sono disponibili presso la Frequent Itemset Mining Dataset Repository (FIMI), una repository dedicata alla raccolta di dataset utilizzati per la valutazione di algoritmi di frequent itemset mining. In particolare, la repository FIMI contiene versioni dei dataset già preprocessate e rappresentate in forma transazionale, ovvero come sequenze di item appartenenti a ciascuna transazione fimi. La scelta di questi dataset è motivata anche dalla necessità di rendere il confronto con il lavoro di Sohrabi e Roshani il più possibile significativo. Gli stessi dataset appartengono infatti all'insieme di benchmark comunemente utilizzati nella letteratura sul frequent itemset mining e sono rappresentativi di caratteristiche differenti in termini di numero di transazioni, numero di item e lunghezza media delle transazioni. Il lavoro di Sohrabi e Roshani valuta il metodo basato sui Cellular Learning Automata confrontandolo con algoritmi tradizionali, tra cui Apriori, FP-Growth e BitTable, considerando differenti valori della soglia di supporto.

L'utilizzo degli stessi dataset permette quindi di effettuare un confronto sperimentale coerente con il contesto di riferimento del metodo implementato. Un aspetto importante riguarda la rappresentazione dei dati. I file messi a disposizione dalla repository FIMI non sono necessariamente accompagnati dalle informazioni semantiche relative alle variabili originarie: il preprocessing effettuato per il frequent itemset mining rappresenta direttamente ogni transazione come un insieme di item. Di conseguenza, le informazioni sul dominio applicativo dei dataset vengono descritte separatamente, quando disponibili, mentre gli algoritmi considerano esclusivamente la rappresentazione transazionale.

2.1 Caratteristiche generali dei dataset

I quattro dataset presentano caratteristiche significativamente differenti. In particolare, Mushroom e PUMSB sono caratterizzati da transazioni relativamente lunghe e da una densità maggiore rispetto a Retail e Kosarak, che presentano invece un numero elevato di item distinti e transazioni mediamente più brevi. Questa varietà rende i dataset adatti a valutare il comportamento degli algoritmi in condizioni differenti. La tabella seguente riassume le principali caratteristiche dei dataset utilizzati. Il numero di item rappresenta il numero di item distinti presenti nel dataset, mentre la lunghezza media della transazione indica il numero medio di item contenuti in una singola transazione.

Dataset	Transazioni	Item	Lunghezza media	Densità
Mushroom	8124	119	23	19,33%
Retail	88162	16470	10	0,06%
PUMSB	49046	2113	74	3,50%
Kosarak	990002	41270	8	0,02%

Table 1: Principali caratteristiche dei dataset utilizzati

La densità può essere interpretata come il rapporto tra il numero medio di item presenti in una transazione e il numero totale di item distinti nel dataset. Un valore elevato indica quindi che le transazioni contengono, mediamente, una porzione maggiore dell'insieme complessivo degli item. Al contrario, nei dataset sparsi una singola transazione contiene generalmente solo una piccola parte degli item disponibili.

2.2 Mushroom

Il dataset Mushroom è caratterizzato da dati relativi ai funghi e nella versione presente nella repository FIMI, è stato trasformato in una rappresentazione adatta al frequent itemset mining. Il dataset contiene 8124 transazioni e 119 item distinti, con una lunghezza media di circa 23 item per transazione. A differenza degli altri dataset considerati, Mushroom permette di fornire una descrizione dettagliata delle variabili originarie, poiché i dati di partenza sono osservazioni descritte attraverso caratteristiche categoriche dei funghi. Le variabili presenti nel dataset originale sono:

Variabile	Descrizione
cap-shape	Forma del cappello
cap-surface	Superficie del cappello
cap-color	Colore del cappello
bruises	Presenza di ammacature
odor	Odore
gill-attachment	Tipo di attacco delle lamelle
gill-spacing	Spaziatura delle lamelle
gill-size	Dimensione delle lamelle
gill-color	Colore delle lamelle
stalk-shape	Forma del gambo
stalk-root	Tipo di radice del gambo
stalk-surface-above-ring	Superficie del gambo sopra l'anello
stalk-surface-below-ring	Superficie del gambo sotto l'anello
stalk-color-above-ring	Colore del gambo sopra l'anello
stalk-color-below-ring	Colore del gambo sotto l'anello
veil-type	Tipo di velo
veil-color	Colore del velo
ring-number	Numero di anelli
ring-type	Tipo di anello
spore-print-color	Colore della sporata
population	Distribuzione della popolazione
habitat	Habitat

Table 2: Variabili originarie del dataset Mushroom

Nella rappresentazione utilizzata dagli algoritmi, le variabili categoriche vengono rappresentate attraverso coppie attributo-valore. Ad esempio, una caratteristica quale `cap-shape=x` viene considerata come un singolo item. Una transazione è quindi costituita dall'insieme degli item corrispondenti alle caratteristiche osservate in una determinata istanza. Questa trasformazione permette di passare dalla rappresentazione originaria del dataset a quella richiesta dagli algoritmi di frequent itemset mining, nei quali ogni transazione è considerata come un insieme di item.

2.3 Retail

Il dataset Retail contiene dati relativi alle transazioni di un supermercato belga anonimo. I dati sono stati raccolti in tre periodi non consecutivi, compresi complessivamente in circa cinque mesi tra la fine del 1999 e il 2000. Il dataset originale contiene 88163 ricevute e 16470 SKU (Stock Keeping Unit) distinti. Nel dataset originale ogni record contiene informazioni relative alla transazione e all'acquisto effettuato. In particolare, sono presenti:

Variabile	Descrizione
date	Data di acquisto
receipt_nr	Identificativo della ricevuta
article_nr	Identificativo dell'articolo
amount	Quantità acquistata
price	Prezzo dell'articolo
customer_nr	Identificativo del cliente

Table 3: Variabili originarie del dataset Retail

La versione utilizzata per il frequent itemset mining non mantiene direttamente tutte queste informazioni, ma rappresenta i dati in forma transazionale. Gli item corrispondono quindi agli articoli acquistati, mentre una transazione rappresenta l'insieme degli articoli associati a una determinata ricevuta. Il dataset Retail presenta una struttura particolarmente sparsa: a fronte di 16470 item distinti, una transazione contiene mediamente circa 10 item. Questa caratteristica permette di analizzare il comportamento degli algoritmi in presenza di un elevato numero di item disponibili e di transazioni relativamente brevi.

2.4 PUMSB

Il dataset PUMSB deriva da dati relativi alla popolazione e alle abitazioni (population and housing census data). La versione disponibile nella repository FIMI è già preprocessata e rappresentata in forma transazionale. Il dataset contiene 49046 transazioni e 2113 item distinti, con una lunghezza media di circa 74 item per transazione. Rispetto a Retail e Kosarak, il dataset presenta transazioni significativamente più lunghe e una maggiore densità. A differenza del dataset Mushroom, nella versione transazionale utilizzata per gli esperimenti gli item sono rappresentati mediante identificativi e non è disponibile, nella repository FIMI, una corrispondenza diretta tra ciascun identificativo e una specifica variabile semantica del dataset originale. Per questo motivo, nell'implementazione gli item di PUMSB vengono trattati esclusivamente come elementi della transazione, senza attribuire loro un significato applicativo specifico.

2.5 Kosarak

Il dataset *Kosarak* è costituito da dati di tipo click-stream anonimizzati provenienti da un portale di news online ungherese. La repository FIMI identifica il dataset come un insieme di dati transazionali ottenuti da tali click-stream. Il dataset contiene circa 990002 transazioni e oltre 41000 item distinti, con una lunghezza media di circa 8 item per transazione. La combinazione tra il numero molto elevato di transazioni, l'elevato numero di item distinti e la ridotta lunghezza media delle transazioni rende Kosarak uno dei dataset più sparsi tra quelli considerati. Gli item presenti nel dataset sono anonimizzati e non possiedono quindi, nella versione distribuita dalla repository FIMI, un'interpretazione semantica direttamente utilizzabile. Di conseguenza, ogni

identificativo viene considerato esclusivamente come un item e l'analisi si concentra sulle relazioni di frequenza tra gli item presenti nelle diverse transazioni.

2.6 Confronto tra i dataset

La scelta dei quattro dataset consente di valutare gli algoritmi in scenari caratterizzati da proprietà differenti. Il dataset Mushroom presenta un numero relativamente ridotto di item e transazioni di lunghezza moderata, mentre PUMSB presenta transazioni mediamente molto più lunghe. I dataset Retail e Kosarak presentano invece una struttura più sparsa, caratterizzata da un numero elevato di item distinti e da transazioni relativamente brevi. Queste differenze sono particolarmente rilevanti nel contesto del frequent itemset mining, poiché la quantità di item disponibili, la lunghezza delle transazioni e la densità del dataset possono influenzare significativamente il numero di candidati generati e di conseguenza, il tempo necessario agli algoritmi per individuare gli itemset frequenti.

3 Algoritmi

In questa sezione vengono descritti gli algoritmi implementati per il problema di frequent itemset mining. L'implementazione è stata realizzata seguendo, per quanto possibile, la struttura e i passaggi descritti nell'articolo di riferimento per il metodo basato sui Cellular Learning Automata e implementando, inoltre, gli algoritmi Apriori e FP-Growth come metodo di paragone. Gli algoritmi ricevono in input un dataset rappresentato in forma transazionale, una soglia minima di supporto e una lunghezza massima degli itemset da individuare. L'obiettivo è ottenere l'insieme degli itemset frequenti, ovvero degli itemset il cui supporto risulta maggiore o uguale alla soglia specificata.

3.1 Cellular Learning Automata

Il primo algoritmo implementato è il metodo basato sui Cellular Learning Automata (CLA), proposto da Sohrabi e Roshani. L'idea alla base del metodo consiste nel rappresentare le relazioni tra gli item attraverso una struttura di celle e liste di prossimità, evitando di generare esplicitamente tutti i candidati possibili come avviene nell'algoritmo Apriori. L'implementazione è organizzata nelle seguenti quattro fasi:

- individuazione degli itemset frequenti di dimensione 1;
- pruning e compressione del dataset;
- costruzione delle celle e delle relative liste di prossimità;
- pruning delle liste di prossimità estrazione degli itemset frequenti.

3.1.1 Classi

Come prima parte dell'implementazione del metodo proposto nell'articolo sono state definite quattro classi:

- **Item:** rappresenta un singolo item e contiene il nome dell'item e il relativo supporto;
- **Transaction:** rappresenta una transazione del dataset. Contiene la lista degli item presenti e un attributo `iteration_number`, utilizzato per indicare quante transazioni si ripete dopo la fase di compressione;
- **Cell:** rappresenta una cella dell'automa ed è caratterizzata dal nome dell'item associato e dalla relativa `proximity_list`;
- **Neighbor:** rappresenta un elemento presente all'interno di una lista di prossimità. Oltre al nome e all'`iteration_number`, un Neighbor può contenere a sua volta una `proximity_list`, permettendo di rappresentare relazioni tra più livelli di item.

3.1.2 Fase 1: individuazione degli itemset frequenti di dimensione 1

La prima fase è implementata dalla funzione `find_1_itemsets(dataset, minsup)`. La funzione inizialmente costruisce un dizionario nel quale, per ogni item presente nel dataset, viene memorizzato il relativo supporto. Il dataset viene quindi scansionato e il conteggio associato a ciascun item viene incrementato ogni volta che l'item viene incontrato. Terminata la scansione, vengono selezionati solamente gli item il cui supporto è maggiore o uguale a `minsup`. Per ciascuno di essi viene creato un oggetto `Item`, contenente il nome dell'item e il relativo supporto. Gli itemset ottenuti vengono infine ordinati alfabeticamente e restituiti dalla funzione.

3.1.3 Fase 2: pruning e compressione del dataset

La funzione `prune_and_compress_dataset(dataset, frequent_itemset)` implementa la seconda fase dell'algoritmo. Lo scopo di questa fase è ridurre la quantità di dati utilizzata nelle operazioni successive. Per prima cosa viene costruito un insieme contenente i nomi degli 1-itemset frequenti. Successivamente, ogni transazione viene analizzata e vengono mantenuti solamente gli item appartenenti a tale insieme. Le transazioni risultanti vengono ordinate e utilizzate come chiavi di un dizionario. In questo modo, due transazioni che dopo il pruning contengono gli stessi item vengono considerate equivalenti anche se originariamente gli item erano presenti in un ordine differente. Quando una stessa transazione compare più volte, invece di memorizzarla nuovamente viene incrementato il relativo `iteration_number`. La funzione restituisce quindi una lista di transazioni compresse, nella quale ogni oggetto rappresenta una particolare combinazione di item e il numero di occorrenze nel dataset originale.

3.1.4 Fase 3: costruzione delle celle e delle liste di prossimità

Le funzioni `run_cellular_automata(compressed_dataset, frequent_itemset)` e `update_proximity_list(neighbor, remaining_items, iteration_number)` implementano la fase centrale del metodo CLA. La prima funzione crea una cella per ciascun itemset frequente di dimensione 1. Ogni cella è quindi identificata dal nome dell'item corrispondente. Per ogni transazione del dataset compresso viene costruita una lista ordinata degli item frequenti presenti nella transazione. L'ordinamento permette di mantenere una rappresentazione coerente delle relazioni tra gli item. Per ciascuna cella presente nella transazione vengono quindi considerati gli altri item della transazione e queste informazioni vengono utilizzate per aggiornare la relativa lista di prossimità. L'aggiornamento delle liste di prossimità viene effettuato dalla seconda funzione. Per ogni item associato alla cella viene creato, se necessario, un oggetto `Neighbor` e viene incrementato il relativo `iteration_number` in base al numero di occorrenze della transazione compressa. La funzione considera inoltre gli item successivi nella lista e memorizza le relazioni tra coppie di elementi all'interno della lista di prossimità del `Neighbor`. Questa struttura ricorsiva permette di rappresentare non solo la presenza di singoli item insieme alla cella, ma anche relazioni tra item a livelli successivi.

3.1.5 Fase 4: pruning delle liste di prossimità e estrazione degli itemset frequenti

Una volta costruite le celle, viene eseguita una nuova fase di pruning attraverso le funzioni `prune_cells(cells, minsup)` e `prune_proximity_list(neighbor, minsup)`. La seconda funzione analizza i `Neighbor` presenti in una lista di prossimità e mantiene solamente quelli il cui `iteration_number` è maggiore o uguale a `minsup`. Per ogni elemento mantenuto, la funzione viene richiamata ricorsivamente sulla relativa lista di prossimità. In questo modo il pruning viene applicato a tutti i livelli della struttura, eliminando le relazioni che non soddisfano il supporto minimo richiesto. La prima funzione applica questo procedimento alla lista di prossimità di ogni cella. Dopo il pruning, gli itemset vengono estratti attraverso le funzioni `scan_cells(cells, max_len)` e `collect_itemsets(neighbor, prefix_item, max_len)`. La funzione `scan_cells()` considera ogni cella come punto di partenza. L'item associato alla cella costituisce un itemset frequente di dimensione 1 e viene successivamente utilizzata la funzione `collect_itemsets()` per costruire itemset di dimensione maggiore. La funzione procede ricorsivamente attraverso le liste di prossimità. Ad ogni livello viene aggiunto al prefisso corrente il nome del `Neighbor` considerato. Se la lunghezza dell'itemset non ha ancora raggiunto il valore massimo `max_len`, la funzione continua ricorsivamente l'esplorazione della relativa lista di prossimità. Poiché lo stesso itemset può essere raggiunto partendo da celle differenti o attraverso ordini differenti degli item, è possibile ottenere duplicati. La funzione `remove_duplicate(itemsets)` risolve questo problema ordinando gli elementi di ciascun itemset, trasformandolo in una tupla e inserendolo in un insieme. In questo modo itemset costituiti dagli

stessi elementi vengono considerati equivalenti indipendentemente dal loro ordine. L'intero procedimento viene coordinato dalla funzione `cla_mining(dataset, minsup, max_len)`, che richiama in sequenza tutte le fasi precedentemente descritte e restituisce la lista finale degli itemset individuati.

3.2 Apriori

Il secondo algoritmo implementato è Apriori, utilizzato come metodo di confronto con il metodo basato sui Cellular Learning Automata. A differenza del metodo CLA, Apriori procede attraverso una generazione iterativa di candidati. A partire dagli itemset frequenti di dimensione 1 vengono generati gli itemset candidati di dimensione successiva. I candidati vengono quindi sottoposti a un controllo basato sulla proprietà di antimonotonicità e, successivamente, viene calcolato il loro supporto attraverso una scansione del dataset.

3.2.1 Fase 1: individuazione degli itemset frequenti di dimensione 1

La funzione `find_1_itemsets_apriori(dataset, minsup)` rappresenta il punto di partenza dell'algoritmo. La funzione scansiona tutte le transazioni e costruisce un dizionario contenente il supporto di ciascun item. Al termine della scansione vengono mantenuti solamente gli item il cui supporto è maggiore o uguale a `minsup`. Gli item frequenti vengono rappresentati come tuple contenenti un singolo elemento e vengono infine ordinati.

3.2.2 Fase 2: generazione dei candidati

La funzione `generate_candidate(frequent_itemsets, target_length)` genera gli itemset candidati. In particolare, la funzione riceve gli itemset frequenti individuati al livello precedente e genera candidati della dimensione richiesta. Gli itemset vengono ordinati e vengono confrontate le coppie di itemset che condividono lo stesso prefisso. Quando la condizione è soddisfatta, i due itemset vengono uniti utilizzando l'unione tra insiemi. Il candidato risultante viene ordinato e trasformato in una tupla. L'utilizzo di un set per memorizzare i candidati permette di evitare duplicati durante la generazione.

3.2.3 Fase 3: verifica della proprietà di antimonotonicità

Dopo la generazione, i candidati vengono filtrati attraverso la funzione `verify_antimonotonicity(candidate_itemset, previous_itemsets)`. La funzione verifica che tutti i sottoinsiemi ottenuti eliminando un singolo elemento dal candidato siano presenti nell'insieme degli itemset frequenti del livello precedente. Se almeno uno di questi sottoinsiemi non è frequente, il candidato viene eliminato. Questa fase sfrutta quindi la proprietà di antimonotonicità: se un itemset è frequente, anche tutti i suoi sottoinsiemi devono essere frequenti. Di conseguenza, la presenza di un sottoinsieme non frequente permette di escludere direttamente il candidato senza doverne calcolare il supporto.

3.2.4 Fase 4: calcolo del supporto dei candidati ed estrazione degli itemset frequenti

La funzione `count_candidates_support(dataset, candidate_itemsets)` calcola il supporto dei candidati rimasti dopo la verifica della proprietà di antimonotonicità. Per ogni transazione, gli item vengono convertiti in un insieme. Successivamente, per ciascun candidato viene verificato se l'insieme dei suoi elementi è un sottoinsieme della transazione attraverso l'operazione `issubset()`. Se il candidato è contenuto nella transazione, il relativo supporto viene incrementato. Al termine della scansione viene restituito un dizionario contenente il supporto di ciascun candidato. La funzione `filter_frequent_itemsets(candidate_support, minsup)` riceve i supporti calcolati per i candidati e mantiene solamente gli itemset che soddisfano la soglia minima di supporto. Gli itemset selezionati vengono ordinati e utilizzati come base per la generazione dei candidati del livello successivo. La funzione `apriori_mining(dataset, minsup, maxlen)` coordina tutte le fasi dell'algoritmo Apriori. Dopo aver individuato gli itemset frequenti di dimensione 1, la funzione procede iterativamente aumentando la dimensione degli itemset fino a `maxlen`. Ad ogni iterazione vengono generati i candidati, viene verificata la proprietà di antimonotonicità, viene calcolato il supporto dei candidati rimasti e vengono selezionati gli itemset frequenti. Se a un determinato livello non vengono individuati itemset frequenti, l'esecuzione viene interrotta anticipatamente. Infine, gli itemset frequenti individuati ai diversi livelli vengono raccolti in un'unica lista che costituisce il risultato dell'algoritmo.

3.3 FP-Growth

Il terzo algoritmo implementato per il confronto con il metodo basato sui Cellular Learning Automata è FP-Growth. Questo algoritmo permette di individuare gli itemset frequenti evitando la generazione esplicita dell'insieme dei candidati, sfruttando una struttura dati compatta denominata FP-Tree (Frequent Pattern Tree). L'implementazione è suddivisa in due fasi principali:

1. costruzione dell'FP-Tree a partire dalle transazioni del dataset;
2. estrazione ricorsiva degli itemset frequenti a partire dall'FP-Tree.

3.3.1 Fase 1: costruzione dell'FP-Tree

La prima fase consiste nella costruzione dell'FP-Tree. Per rappresentare i nodi dell'albero viene definita la classe `FPTreeNode`. Ogni nodo rappresenta un item e contiene le informazioni necessarie per descrivere la struttura gerarchica dell'albero. In particolare, ciascun nodo presenta i seguenti attributi:

- `name`, identifica l'item rappresentato dal nodo;
- `support`, memorizza il supporto associato al nodo;
- `parent`, contiene un riferimento al nodo padre;

- `children`, un dizionario contenente i nodi figli del nodo corrente, indicizzati in base al nome dell'item.

L'implementazione della costruzione dell'albero è suddivisa nelle due seguenti funzioni: `find_1_itemsets_fpgrowth(transactions_counts, minsup)` per l'estrazione degli itemset frequenti di dimensione 1 e `build_fp_tree(transactions_counts, minsup)` per la costruzione vera e propria dell'albero. La prima funzione viene utilizzata per individuare gli item frequenti di dimensione 1 e per calcolare il supporto di tutti gli item presenti nel dataset. Per ogni transazione viene considerato ciascun item in essa contenuto e il relativo supporto viene incrementato in base al conteggio associato alla transazione. Al termine di questa fase vengono mantenuti esclusivamente gli item il cui supporto è maggiore o uguale alla soglia minima `minsup`. La funzione restituisce quindi la lista degli item frequenti e un dizionario contenente il supporto di ciascun item. La seconda funzione utilizza queste informazioni per costruire l'FP-Tree. Gli item frequenti vengono inizialmente ordinati in ordine decrescente rispetto al loro supporto. In caso di supporto uguale viene utilizzato il nome dell'item come criterio secondario di ordinamento. Questo ordinamento consente alle transazioni che condividono gli item più frequenti di condividere anche i prefissi dei relativi percorsi nell'albero, permettendo così una rappresentazione compatta delle transazioni. Successivamente viene costruita la header table. Essa è rappresentata da un dizionario che associa ogni item frequente alla lista dei nodi dell'FP-Tree che rappresentano quell'item. La header table può quindi essere considerata come un indice che permette di individuare rapidamente tutte le occorrenze di uno specifico item nell'albero. Viene quindi inizializzata la radice dell'FP-Tree, che non rappresenta alcun item. Per ogni transazione vengono eliminati gli item non frequenti e gli item rimanenti vengono ordinati secondo lo stesso criterio utilizzato in precedenza. La transazione filtrata e ordinata viene successivamente inserita nell'albero a partire dalla radice. Per ogni item della transazione viene verificato se esiste già, tra i figli del nodo corrente, un nodo associato allo stesso item. Se il nodo esiste, esso viene riutilizzato e il suo supporto viene incrementato del conteggio associato alla transazione. In caso contrario viene creato un nuovo nodo, collegato al proprio padre e aggiunto sia alla struttura `children` del padre sia alla lista corrispondente nella header table. Il riutilizzo dei nodi permette a transazioni che condividono lo stesso prefisso di condividere lo stesso percorso nell'albero. L'FP-Tree costituisce quindi una rappresentazione compatta del database delle transazioni, nella quale le informazioni relative agli item frequenti vengono aggregate attraverso percorsi comuni.

3.3.2 Fase 2: estrazione degli itemset frequenti

Una volta costruito l'FP-Tree, la seconda fase dell'algoritmo consiste nell'estrazione degli itemset frequenti con le funzioni `build_conditional_pattern_base(header_table, item_name)` e `mine_fp_tree(transactions_counts, minsup, max_len, prefix_items)`. La prima funzione costruisce la conditional pattern base relativa a uno specifico item. Per ogni nodo associato all'item considerato nella header table, si risale

l'albero attraverso i riferimenti al nodo padre, partendo dal padre del nodo corrente e procedendo verso la radice. Durante questa risalita vengono raccolti i nomi degli item presenti nei nodi attraversati. La radice viene esclusa poiché non rappresenta alcun item. Il percorso ottenuto rappresenta quindi il prefisso che conduce al nodo dell'item considerato. Se il percorso contiene almeno un item, esso viene inserito nella conditional pattern base insieme al supporto del nodo di partenza. L'estrazione vera e propria degli itemset viene effettuata dalla seconda funzione. La funzione costruisce innanzitutto un nuovo FP-Tree a partire dalle transazioni ricevute. Gli item presenti nella header table vengono quindi ordinati in ordine crescente rispetto al loro supporto. Per ogni item viene costruito un nuovo itemset, indicato come `current_itemset`, ottenuto aggiungendo l'item corrente agli eventuali item già presenti nel prefisso `prefix_items`. L'itemset così ottenuto viene aggiunto all'insieme dei pattern individuati. Se la lunghezza dell'itemset non ha ancora raggiunto il valore massimo `max_len`, viene costruita la conditional pattern base relativa all'item corrente. Quest'ultima viene utilizzata come input per una nuova chiamata ricorsiva di `mine_fp_tree`, passando come nuovo prefisso l'itemset appena costruito. La procedura viene quindi ripetuta ricorsivamente, consentendo di individuare itemset di dimensione progressivamente maggiore. La ricorsione termina quando viene raggiunta la lunghezza massima specificata da `max_len` oppure quando non è più possibile costruire ulteriori pattern frequenti. La funzione `fpgrowth_mining(dataset, minsup, max_len)` rappresenta il punto di ingresso dell'implementazione. In sintesi, l'implementazione segue quindi il principio fondamentale di FP-Growth: il database viene dapprima compresso attraverso la costruzione dell'FP-Tree e, successivamente, l'estrazione degli itemset frequenti viene effettuata analizzando ricorsivamente le conditional pattern base associate ai diversi item. In questo modo non è necessario generare esplicitamente tutti i possibili itemset candidati, riducendo il numero di operazioni necessarie per l'estrazione dei pattern frequenti.

4 Analisi dei risultati

In questa sezione vengono analizzati i risultati ottenuti dal confronto tra i tre algoritmi considerati: CLA, Apriori e FP-Growth. Il confronto è stato effettuato su quattro dataset diversi: Mushroom, Retail, Pumsb e Kosarak. Per ciascun dataset sono stati considerati quattro valori della soglia minima di supporto:

$$minsup \in \{0.02, 0.04, 0.06, 0.08\}$$

Per ogni configurazione sono stati misurati i tempi di esecuzione dei tre algoritmi e sono stati confrontati gli itemset frequenti trovati. In tutti i casi gli algoritmi hanno restituito gli stessi itemset. Questo significa che i tre algoritmi producono esattamente gli stessi risultati e che le differenze osservate riguardano solamente il tempo necessario per ottenere tali risultati.

4.1 Dataset Mushroom

Nel dataset Mushroom si osservano differenze molto significative tra i tre algoritmi, soprattutto tra FP-Growth e gli altri due metodi.

- Per $\text{minsup} = 0.02$ vengono trovati 22513 itemset frequenti. CLA impiega 12.0354 secondi, Apriori 75.6909 secondi, mentre FP-Growth impiega solamente 0.7844 secondi. FP-Growth risulta quindi nettamente più veloce rispetto agli altri due algoritmi.
- Per $\text{minsup} = 0.04$ il numero di itemset frequenti diminuisce a 13671. I tempi di esecuzione sono 14.5144 secondi per CLA, 45.4468 secondi per Apriori e 1.1182 secondi per FP-Growth. Anche in questo caso FP-Growth presenta il tempo di esecuzione più basso.
- Per $\text{minsup} = 0.06$ vengono trovati 9705 itemset frequenti. CLA impiega 10.1515 secondi, Apriori 32.9716 secondi e FP-Growth 0.4384 secondi. Anche in questa configurazione il vantaggio di FP-Growth è molto evidente.
- Per $\text{minsup} = 0.08$ vengono trovati 6605 itemset frequenti. I tempi sono 7.2364 secondi per CLA, 22.5916 secondi per Apriori e 0.3106 secondi per FP-Growth.

Dal grafico relativo al dataset Mushroom si può quindi osservare che FP-Growth è sempre l'algoritmo più veloce. Apriori è invece sempre il più lento, mentre CLA si trova in una posizione intermedia. Un aspetto interessante è la diminuzione del numero di itemset frequenti all'aumentare di minsup . Si passa infatti da 22513 itemset con $\text{minsup} = 0.02$ a 6605 itemset con $\text{minsup} = 0.08$. L'aumento della soglia minima di supporto permette quindi di eliminare un numero maggiore di itemset non frequenti e, in generale, comporta una riduzione del tempo di esecuzione. Il comportamento di FP-Growth è particolarmente positivo su questo dataset. Il numero elevato di itemset frequenti rende vantaggiosa la rappresentazione compatta delle transazioni attraverso l'FP-Tree, permettendo di ottenere un tempo di esecuzione inferiore rispetto ad Apriori e CLA.

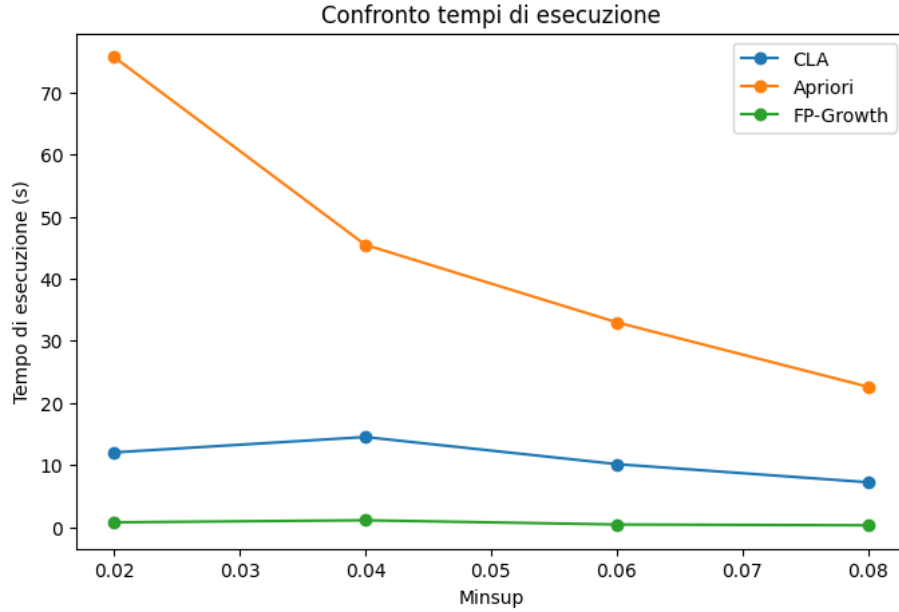


Figure 1: Tempi di esecuzione sul dataset Mushroom

4.2 Dataset Retail

Il comportamento del dataset Retail è diverso rispetto a quello osservato per Mushroom. In questo caso il numero di itemset frequenti è molto più basso e le differenze tra i tre algoritmi sono meno marcate.

- Per $\text{minsup} = 0.02$ vengono trovati 54 itemset frequenti. CLA impiega 0.3353 secondi, Apriori 3.6296 secondi e FP-Growth 0.7293 secondi. In questa configurazione CLA è quindi l'algoritmo più veloce, mentre Apriori è il più lento.
- Per $\text{minsup} = 0.04$ vengono trovati 18 itemset. I tempi sono 0.3969 secondi per CLA, 1.2865 secondi per Apriori e 0.5386 secondi per FP-Growth. Anche in questo caso CLA risulta più veloce di FP-Growth.
- Per $\text{minsup} = 0.06$ vengono trovati 15 itemset. In questa configurazione FP-Growth diventa l'algoritmo più veloce, con un tempo di 0.2983 secondi. CLA impiega 0.4019 secondi, mentre Apriori impiega 0.5707 secondi.
- Per $\text{minsup} = 0.08$ vengono trovati solamente 13 itemset. CLA impiega 0.2262 secondi, FP-Growth 0.2915 secondi e Apriori 0.5136 secondi. CLA torna quindi ad essere l'algoritmo più veloce.

Il dataset Retail mostra quindi che FP-Growth non è necessariamente il metodo più veloce in ogni situazione. Quando il numero di itemset frequenti è molto

basso, il vantaggio derivante dalla rappresentazione tramite FP-Tree può essere ridotto. Il costo necessario per costruire l'albero e per effettuare l'estrazione ricorsiva dei pattern può infatti diventare rilevante rispetto al tempo complessivo di esecuzione. In tutte le configurazioni Apriori risulta comunque più lento rispetto agli altri due algoritmi. Tuttavia, le differenze sono molto più contenute rispetto a quelle osservate nel dataset Mushroom.

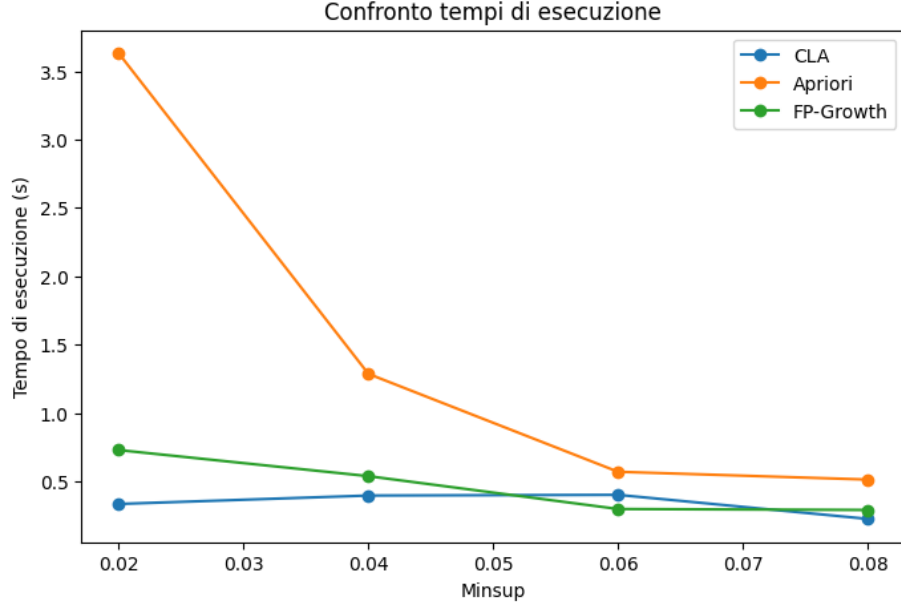


Figure 2: Tempi di esecuzione sul dataset Retail

4.3 Dataset Pumsb

Il dataset Pumsb presenta i tempi di esecuzione più elevati tra tutti i dataset considerati. Questo comportamento è legato soprattutto all'elevato numero di itemset frequenti individuati.

- Per $\text{minsup} = 0.02$ vengono trovati 455351 itemset frequenti. CLA impiega 3448.6976 secondi, Apriori 10154.0501 secondi e FP-Growth 600.8944 secondi. I tempi corrispondono rispettivamente a circa 57.5 minuti, 169.2 minuti e 10 minuti. In questa configurazione FP-Growth è quindi nettamente più veloce rispetto agli altri due algoritmi. La differenza è particolarmente evidente nel confronto con Apriori.
- Per $\text{minsup} = 0.04$ il numero di itemset frequenti diminuisce a 276889. CLA impiega 3157.18 secondi, Apriori 6666.2923 secondi e FP-Growth 456.5612 secondi.

- Per $\text{minsup} = 0.06$ vengono trovati 208506 itemset frequenti. I tempi sono 2728.0567 secondi per CLA, 4994.3771 secondi per Apriori e 359.0799 secondi per FP-Growth.
- Per $\text{minsup} = 0.08$ vengono trovati 167297 itemset frequenti. CLA impiega 2422.9886 secondi, Apriori 4141.1949 secondi e FP-Growth 291.4345 secondi.

Anche osservando il grafico relativo a Pumsb si nota quindi che FP-Growth è sempre il metodo più veloce. Apriori è sempre il più lento, mentre CLA presenta tempi intermedi. Il risultato più importante per questo dataset è la grande differenza tra i tempi di esecuzione. Anche aumentando minsup fino a 0.08, vengono ancora trovati 167297 itemset frequenti. Di conseguenza, il problema rimane molto costoso dal punto di vista computazionale. In questo caso il vantaggio della struttura FP-Tree risulta particolarmente evidente. FP-Growth riesce, infatti, a gestire un numero molto elevato di itemset frequenti con tempi significativamente inferiori rispetto agli altri due algoritmi.

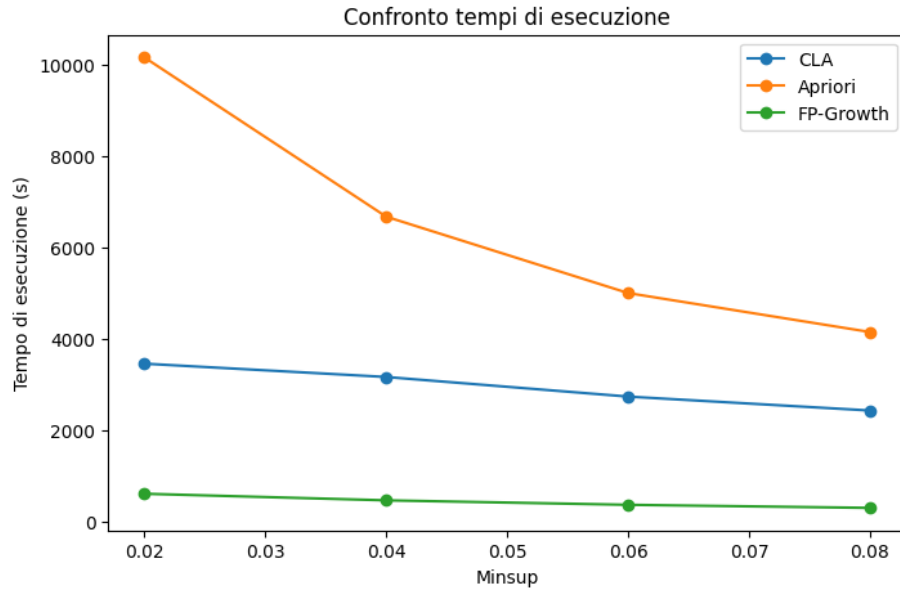


Figure 3: Tempi di esecuzione sul dataset Pumsb

4.4 Dataset Kosarak

Il dataset Kosarak presenta un comportamento differente rispetto a Mushroom e Pumsb. FP-Growth risulta il più veloce quando minsup è pari a 0.02, mentre per valori maggiori della soglia CLA presenta tempi inferiori.

- Per $\text{minsup} = 0.02$ vengono trovati 106 itemset frequenti. CLA impiega 8.9369 secondi, Apriori 97.9181 secondi e FP-Growth 6.9927 secondi. In

questo caso FP-Growth è quindi l'algoritmo più veloce.

- Per $\text{minsup} = 0.04$ il numero di itemset frequenti diminuisce a 41. CLA impiega 3.1269 secondi, Apriori 25.4608 secondi e FP-Growth 4.6465 secondi. In questa configurazione CLA diventa il metodo più veloce.
- Per $\text{minsup} = 0.06$ vengono trovati 24 itemset. I tempi sono 3.8919 secondi per CLA, 15.5135 secondi per Apriori e 4.5499 secondi per FP-Growth. Anche in questo caso CLA risulta leggermente più veloce di FP-Growth.
- Per $\text{minsup} = 0.08$ vengono trovati 14 itemset frequenti. CLA impiega 2.6784 secondi, Apriori 8.5597 secondi e FP-Growth 4.2968 secondi.

Il comportamento di Kosarak mostra quindi che il metodo più veloce può dipendere sia dal dataset sia dal valore della soglia minima di supporto. Per $\text{minsup} = 0.02$ FP-Growth presenta il tempo migliore, mentre per i valori successivi CLA risulta più veloce. Una possibile spiegazione è che, quando il numero di itemset frequenti è ridotto, il vantaggio derivante dalla costruzione dell'FP-Tree può diminuire. In queste situazioni il costo aggiuntivo necessario per costruire l'albero e analizzare ricorsivamente le conditional pattern base può rendere FP-Growth meno conveniente rispetto a CLA. Anche in questo dataset Apriori presenta sempre tempi superiori rispetto agli altri due algoritmi.

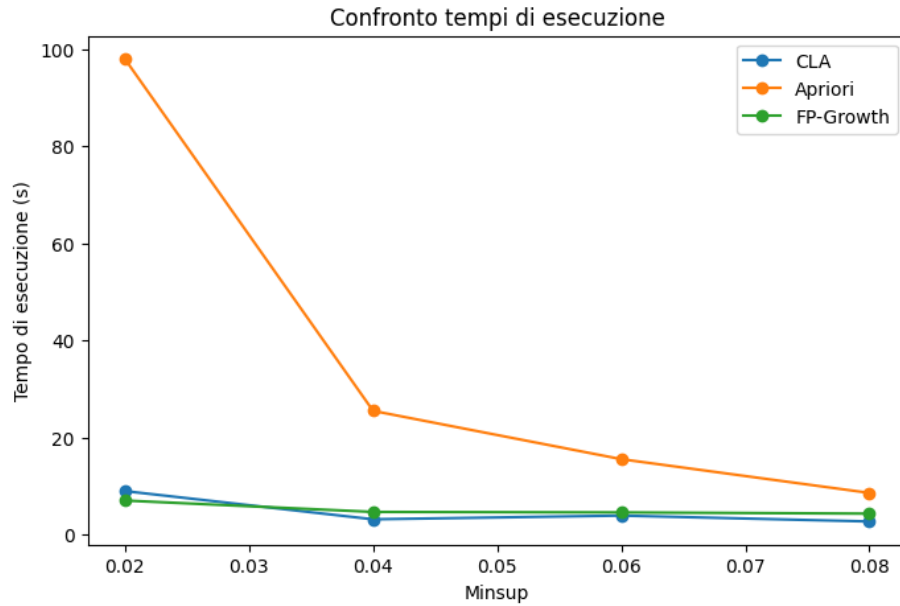


Figure 4: Tempi di esecuzione sul dataset Kosarak

4.5 Confronto complessivo

Considerando complessivamente i risultati dei quattro dataset, emerge che non esiste un algoritmo che risulti sempre il più veloce indipendentemente dal dataset e dal valore di minsup. FP-Growth presenta i risultati migliori soprattutto nei dataset in cui viene prodotto un numero elevato di itemset frequenti. Questo comportamento è particolarmente evidente nei dataset Mushroom e Pumsb. Nel caso di Mushroom, FP-Growth è sempre il metodo più veloce e presenta tempi molto inferiori rispetto a CLA e Apriori. Nel caso di Pumsb la differenza è ancora più significativa in termini assoluti, poiché tutti gli algoritmi devono elaborare un numero molto elevato di itemset. Il dataset Retail presenta invece un numero molto più limitato di itemset frequenti. In questo caso CLA risulta spesso più veloce di FP-Growth. La differenza tra i tempi è comunque contenuta e tutti gli algoritmi completano l'esecuzione in pochi secondi. Kosarak presenta un comportamento intermedio. FP-Growth è il più veloce per minsup = 0.02, mentre CLA diventa più veloce per le soglie successive. Questo risultato mostra che le prestazioni degli algoritmi dipendono non solo dalla dimensione del dataset, ma anche dalla quantità di pattern frequenti che devono essere estratti. Per quanto riguarda Apriori, dai risultati emerge un comportamento abbastanza uniforme: l'algoritmo risulta generalmente il più lento, soprattutto quando il numero di itemset frequenti è elevato. La differenza rispetto agli altri algoritmi è particolarmente evidente nei dataset Mushroom e Pumsb. Un altro risultato importante riguarda l'effetto della soglia minima di supporto. In generale, aumentando minsup diminuisce il numero di itemset frequenti. Questo fenomeno è particolarmente evidente nel dataset Pumsb, dove si passa da 455351 itemset con minsup = 0.02 a 167297 itemset con minsup = 0.08. Anche nel dataset Mushroom si osserva una riduzione significativa, da 22513 a 6605 itemset. La riduzione del numero di itemset porta generalmente a una riduzione del tempo di esecuzione, anche se il comportamento non è sempre perfettamente regolare. Infatti, il tempo dipende non soltanto dal numero finale di itemset trovati, ma anche dalla struttura del dataset e dalle operazioni necessarie per costruire ed esplorare le strutture utilizzate dai diversi algoritmi. Infine, il controllo effettuato sugli itemset permette di verificare che le differenze nei tempi non siano dovute a una diversa quantità o qualità dei risultati. Per tutte le configurazioni considerate, i tre algoritmi hanno prodotto esattamente gli stessi itemset frequenti. Questo conferma quindi la correttezza delle implementazioni e permette di concentrarsi esclusivamente sul confronto dell'efficienza computazionale. In conclusione, i risultati mostrano che FP-Growth è particolarmente efficace quando il numero di pattern frequenti è elevato, come nei dataset Mushroom e Pumsb. CLA risulta invece competitivo, e in alcuni casi più veloce, quando il numero di pattern è più contenuto, come nei dataset Retail e Kosarak. Apriori presenta complessivamente le prestazioni peggiori, soprattutto nei casi in cui lo spazio degli itemset frequenti diventa molto grande. Seguendo l'idea che viene proposta nell'articolo si potrebbe pensare di combinare CLA e FP-Growth per unire l'efficienza dei due metodi.

References

- [1] Sohrabi and Roshani. Frequent itemset mining using cellular learning automata. 2016.